

03

강

R 컴퓨팅

데이터 불러오기와 저장하기

통계·데이터과학과 장영재 교수

학습목차

1 데이터 입력

2 데이터 불러오기

3 데이터 저장하기

4 데이터 구조의 확인 및 결측치 지정

5 객체 확인 및 삭제

6 실습하기

학습목표

- 1 R 콘솔 상에서 데이터를 입력할 수 있다.
- 2 외부 데이터 파일을 불러올 수 있다.
- 3 분석한 결과를 외부 파일로 저장할 수 있다.
- 4 작업 경로를 지정하고 확인할 수 있다.
- 5 생성된 객체를 확인하고 삭제할 수 있다.





데이터 입력

1. 데이터 입력

R 콘솔에서는 직접 키보드를 이용하여 데이터를 입력할 수 있으며, 입력되어 생성된 데이터를 객체명을 지정하여 저장하면 현 작업 환경이 종료되기 전에는 객체 이름으로 호출 가능

▶ c() 함수를 이용한 입력

- c() 함수는 가장 기본적인 데이터 입력방법(concatenate 또는 combine의 첫 글자)
- 좌측에 벡터 이름(객체명)이 위치하며, '<-' 나 '=' 연산자는 데이터가 할당됨을 의미하고, 우측에는 입력대상이 되는 데이터의 관측값이 c 함수 내에 나타나게 됨

예제 2-1 1,2,3,4,5의 5개 관측값을 갖는 벡터 x를 생성

```
> x <- c(1,2,3,4,5)
```

```
> x
```

```
[1] 1 2 3 4 5
```

▶ 예제 2-2 6, 7, 8, 9, 10의 5개 관측값을 갖는 벡터 y를 생성하되 입력결과를 바로 화면에 출력

```
> (y <- c(6,7,8,9,10))
```

```
[1] 6 7 8 9 10
```

1. 데이터 입력

예제 2-3 a, b, c의 3개 관측값을 갖는 벡터 x1과 T, F, T, T 4개 관측값을 갖는 벡터 x2를 생성

```
> (x1 <- c("a","b","c"))
```

```
[1] "a" "b" "c"
```

```
> (x2 <- c(T,F,T,T))
```

```
[1] TRUE FALSE TRUE TRUE
```

▶ scan() 함수를 이용한 데이터 입력

- R에서는 scan() 함수를 활용하여 바로 데이터를 입력할 수 있음
- 함수를 실행하면 실행 프롬프트가 "1:"으로 바뀌게 되는데 이때 자료를 하나씩 입력하고 Enter 키를 치면 다음 값을 입력할 수 있게 됨

1. 데이터 입력

예제 2-4 scan() 함수를 이용하여 11, 12, 13, 14, 15의 5개 관측값을 갖는 벡터 z1 생성

```
> z1 <- scan()
```

```
1: 11
```

```
2: 12
```

```
3: 13 14 15
```

```
6:
```

```
Read 5 items
```

```
> z1
```

```
[1] 11 12 13 14 15
```

- 함수의 인수에 what="character" 라고 하거나, what=" "라고 하면 문자형 데이터로 인식

1. 데이터 입력

예제 2-5 scan() 함수를 이용하여 A, B, C의 3개 관측값을 갖는 벡터 z2를 생성

```
> z2 = scan( what=" " )
```

```
1: A
```

```
2: B
```

```
3: C
```

```
4:
```

```
Read 3 items
```

```
> z2
```

```
[1] "A" "B" "C"
```

1. 데이터 입력

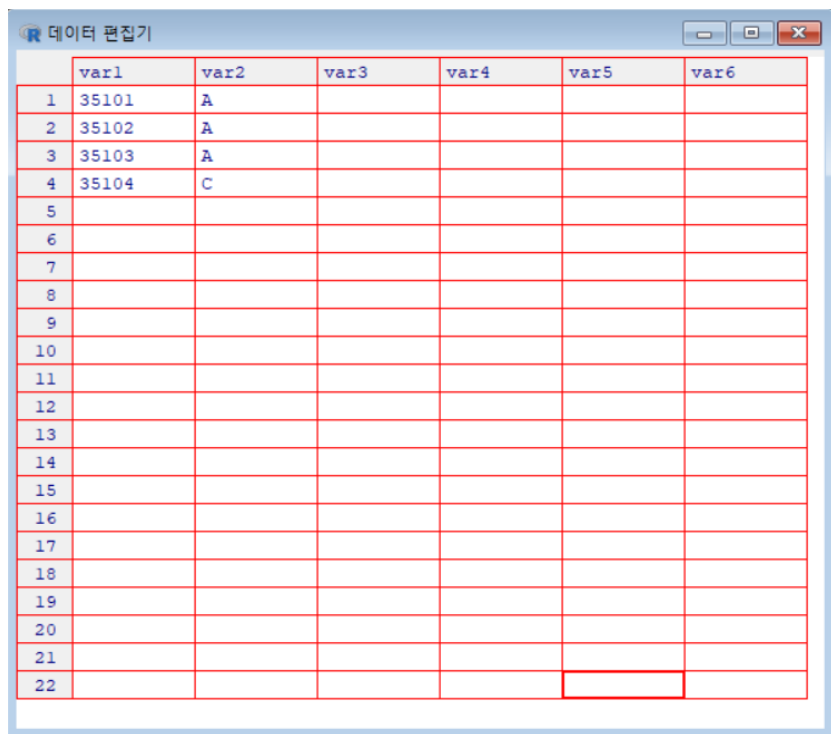
▶ edit() 함수를 이용한 입력

예제 2-6 edit() 함수를 이용하여 데이터 프레임을 생성하기 위해 빈 객체를 데이터 프레임으로 지정해 준 뒤

창을 열어 데이터를 입력

```
> dat1 <- data.frame()
```

```
> dat1 = edit(dat1)
```



	var1	var2	var3	var4	var5	var6
1	35101	A				
2	35102	A				
3	35103	A				
4	35104	C				
5						
6						
7						
8						
9						
10						
11						
12						
13						
14						
15						
16						
17						
18						
19						
20						
21						
22						

<데이터 편집기 창 입력>

1. 데이터 입력

```
> dat1
```

```
var1 var2
```

```
1 35101 A
```

```
2 35102 A
```

```
3 35103 A
```

```
4 35104 C
```



데이터 불러오기

2. 데이터 불러오기

▶ read.table() 함수를 이용한 데이터 불러오기

- read.table() 함수를 이용하면 스프레드시트 형태의 외부 파일을 불러올 수 있음
- 행과 열로 구성된 2차원 구조를 가지며 각 행은 하나의 관측치를 나타내고 각 열은 변수 값을 의미
- 관측치 내의 변수에 해당하는 값들은 공백, 탭, 콜론(:), 콤마(,) 등의 구분기호로 구분

예제 2-7 "C:/data/"와 같은 경로에 저장된 example1.txt라는 파일(첫 행에는 변수명, 각 관측치 내 변수 값들은 공백으로 구분)을 read.table() 함수를 이용하여 불러오기

```
> read.table("C:/data/example1.txt", header=T)
```

```
name class subject
```

```
1 Chang 1 Math
```

```
2 Choi 1 English
```

```
3 Kim 2 Science
```

```
4 Lee 2 Math
```

```
5 Park 3 History
```

2. 데이터 불러오기

- " " 또는 ' '를 이용해서파일이 위치한 경로와 파일이름을 정확하게 지정
- header 옵션 지정에 따라 첫째 줄의 관측치를 변수명으로 인식하느냐 여부가 결정
- 변수 값이 콤마(,)로 구분되어 있는 경우, sep="," 옵션을 사용

예제 2-8 "C:/data/"와 같은 경로에 저장된 example2.txt라는 파일(첫 행에는 변수명, 각 관측치 내 변수 값들은 콤마(,)로 구분)을 read.table() 함수를 이용하여 불러오기

```
name,class,subject
```

```
Chang,1,Math
```

```
Choi,1,English
```

```
Kim,2,Science
```

```
Lee,2,Math
```

```
Park,3,History
```

<example2.txt 파일>

```
> read.table("C:/data/example2.txt", header=T, sep=",")
```

```
name class subject
```

```
1 Chang 1 Math
```

```
2 Choi 1 English
```

```
3 Kim 2 Science
```

```
4 Lee 2 Math
```

```
5 Park 3 History
```

2. 데이터 불러오기

▶ 작업 폴더(작업 경로)의 지정

- 작업 경로를 지정할 때는 윈도우 환경에서도 디렉토리 구분 기호를 "/" 또는 "\\"로 지정

📁 현 작업 경로를 확인하려면 `getwd()` 함수를 사용

📁 작업 경로 지정은 기본적으로 현 작업 세션에만 적용되므로 매번 `setwd()` 함수를 이용해 지정

📁 환경 저장 옵션을 사용하여 .RData 파일이나 R 프로젝트 설정에 작업 경로를 포함시키면,

이후 세션에서도 동일한 경로가 자동으로 유지

예제 2-9 "C:/data/"와 같은 경로에 저장된 example1.txt라는 파일(첫 행에는 변수명이 기록됨)이 있음.

작업 폴더(작업 경로)를 지정한 뒤 이 파일을 불러와서 dat2라는 이름의 객체로 저장

```
> setwd("C:/data")
```

```
> dat2 <- read.table("example1.txt", header=T)
```

2. 데이터 불러오기

▶ read.csv() 함수를 이용한 데이터 불러오기

- CSV는 Comma Separated Values의 약어로, 콤마(,)를 구분기호로 사용하는 데이터 형태를 의미
- read.csv() 함수를 이용하면 CSV 형태의 파일을 불러올 수 있음(sep=","와 header=TRUE가 디폴트)

예제 2-10 read.csv() 함수를 이용하여 현 작업경로에 저장되어 있는 "example2.txt"를 불러와서

dat3라는 객체로 저장

```
> dat3 <- read.csv("example2.txt")
```

```
> dat3
```

```
name class subject
```

```
1 Chang 1 Math
```

```
2 Choi 1 English
```

```
3 Kim 2 Science
```

```
4 Lee 2 Math
```

```
5 Park 3 History
```

2. 데이터 불러오기

예제 2-11 현 작업경로에 USArrests1.csv라는 데이터세트가 저장되어 있다고 가정할 때,

read.csv() 함수를 이용하여 USArrests1.csv를 불러오고 us_dat라는 객체명으로 저장

```
> us_dat <- read.csv("USArrests1.csv")
```

```
➤ head(us_dat)
```

▶ scan() 함수를 이용한 데이터 불러오기

- scan() 함수는 콘솔에서 직접 데이터를 입력하는 데 자주 사용되지만, 외부 파일로부터

데이터를 읽어들이는 기능도 수행

- 기본값은 숫자형(numeric)이므로, 파일에 문자형 데이터가 포함되어 있다면 what 옵션을 설정

2. 데이터 불러오기

예제 2-12 scan() 함수를 이용하여 “example2.txt”를 불러와서 dat4라는 객체로 저장

```
> dat4 <- scan("example2.txt", what=list(name="",class=0,subject=""),skip=1,sep=",")
```

Read 5 records

```
> dat4
```

\$name

```
[1] "Chang" "Choi" "Kim" "Lee" "Park"
```

\$class

```
[1] 1 1 2 2 3
```

\$subject

```
[1] "Math" "English" "Science" "Math" "History"
```

▶ 웹사이트의 데이터 불러오기

- read.table() 등의 함수를 사용할 때, 파일 이름 대신 웹사이트 상의 파일 주소(URL)를 지정하면 불러올 수 있음

2. 데이터 불러오기

예제 2-13 UCI 저장소에서 제공하는 auto-mpg.data 파일 불러오기

<https://archive.ics.uci.edu/ml/machine-learning-databases/auto-mpg/auto-mpg.data>

```
> url <- "https://archive.ics.uci.edu/ml/machine-learning-databases/auto-mpg/auto-mpg.data"
```

```
> mpg_data <- read.table(url, header = FALSE, na.strings = "?",  
+ col.names = c("mpg", "cylinders", "displacement", "horsepower",  
+ "weight", "acceleration", "year", "origin", "car_name"))
```

```
> head(mpg_data)
```

- 자동차의 연비(mpg), 실린더 수(cylinders), 배기량(displacement), 마력(horsepower), 중량(weight), 가속능력(acceleration), 제조 연도(year), 생산지(origin), 그리고 자동차 이름(car_name) 변수를 포함
- na.strings = "?" 옵션을 지정한 것은 데이터의 일부 결측치가 "?"로 표시되어 있기 때문

03

데이터 저장하기

3. 데이터 저장하기

- ▶ sink() 함수를 활용한 출력 결과 파일 저장
 - 함수 sink()를 활용하여 화면에 출력된 모든 결과를 파일로 저장

예제 2-14 sink() 함수를 이용하여 R의 기본 패키지에 내장된 데이터 중 cars라는 데이터세트의 기술통계량을 외부 파일 sinkexample.txt로 저장

```
> sink('sinkexample.txt')  
> summary(cars)  
> sink()
```

3. 데이터 저장하기

윈도우 탐색기에서 현 작업 경로 내의 sinkexample.txt를 메모장에서 열면 다음과 같은 summary 결과가 저장됨을 확인할 수 있다.

speed dist

Min. : 4.0 Min. : 2.00

1st Qu.:12.0 1st Qu.: 26.00

Median :15.0 Median : 36.00

Mean :15.4 Mean : 42.98

3rd Qu.:19.0 3rd Qu.: 56.00

Max. :25.0 Max. :120.00

3. 데이터 저장하기

▶ cat() 함수를 활용한 출력 결과 파일 저장

- cat() 함수는 여러 개의 객체를 한 번에 출력하여 파일로 저장하는 기능을 수행
- sink() 함수가 콘솔 전체 출력을 파일로 기록하는 것과 달리, cat() 함수는 사용자가 지정한 문자열과 객체를 결합해 줄 단위로 저장)

3. 데이터 저장하기

예제 2-15 두 개의 벡터 (10, 20, 30, 40, 50)와 (5, 4, 3, 2, 1)를 만들고, 두 벡터의 사칙연산 결과를 cat() 함수를 이용하여

현 작업경로에 외부 파일로 저장

```
> setwd("C:/data")  
> a <- c(10, 20, 30, 40, 50)  
> b <- c(5, 4, 3, 2, 1)  
> sum_ab <- a + b  
> diff_ab <- a - b  
> prod_ab <- a * b  
> div_ab <- a / b  
> cat("Vector a = (", a, ")", "\n", file = "calc_results.txt")  
> cat("Vector b = (", b, ")", "\n", file = "calc_results.txt", append = TRUE)  
> cat("a + b = (", sum_ab, ")", "\n", file = "calc_results.txt", append = TRUE)  
> cat("a - b = (", diff_ab, ")", "\n", file = "calc_results.txt", append = TRUE)  
➤ cat("a * b = (", prod_ab, ")", "\n", file = "calc_results.txt", append = TRUE)  
> cat("a / b = (", div_ab, ")", "\n", file = "calc_results.txt", append = TRUE)
```

3. 데이터 저장하기

▶ write.table() 함수를 이용한 데이터 저장

- 데이터를 외부로 저장하는 대표적인 함수로 write.table() 함수가 있다. 이 함수와 write.csv() 함수는 R의 객체를 외부로 저장한다는 점에서는 유사하지만, 기본적으로 지정되어 있는 옵션상 차이

예제 2-16 R의 기본 패키지에 저장된 데이터 pressure를 write.table() 함수를 이용하여 다음과 같이 여러 개의 외부 파일로 저장

```
> write.table( pressure, "pressure1.txt" )
```

```
> write.table( pressure, "pressure2.txt", row.names=FALSE )
```

```
> write.table( pressure, "pressure3.txt", row.names=FALSE, quote=FALSE )
```

```
> write.table( pressure, "pressure4.txt", row.names=FALSE, quote=FALSE, sep="\t" )
```

```
> write.table( pressure, "pressure5.txt", row.names=FALSE, quote=FALSE, sep="," )
```

▶ write.csv() 함수를 이용한 데이터 저장

- CSV 형식 저장에 최적화되어 있어 보다 간편하게 사용할 수 있으며 데이터 프레임을

CSV(Comma-Separated Values) 형식으로 저장하므로, 저장된 파일의 범용성이 높은 특징

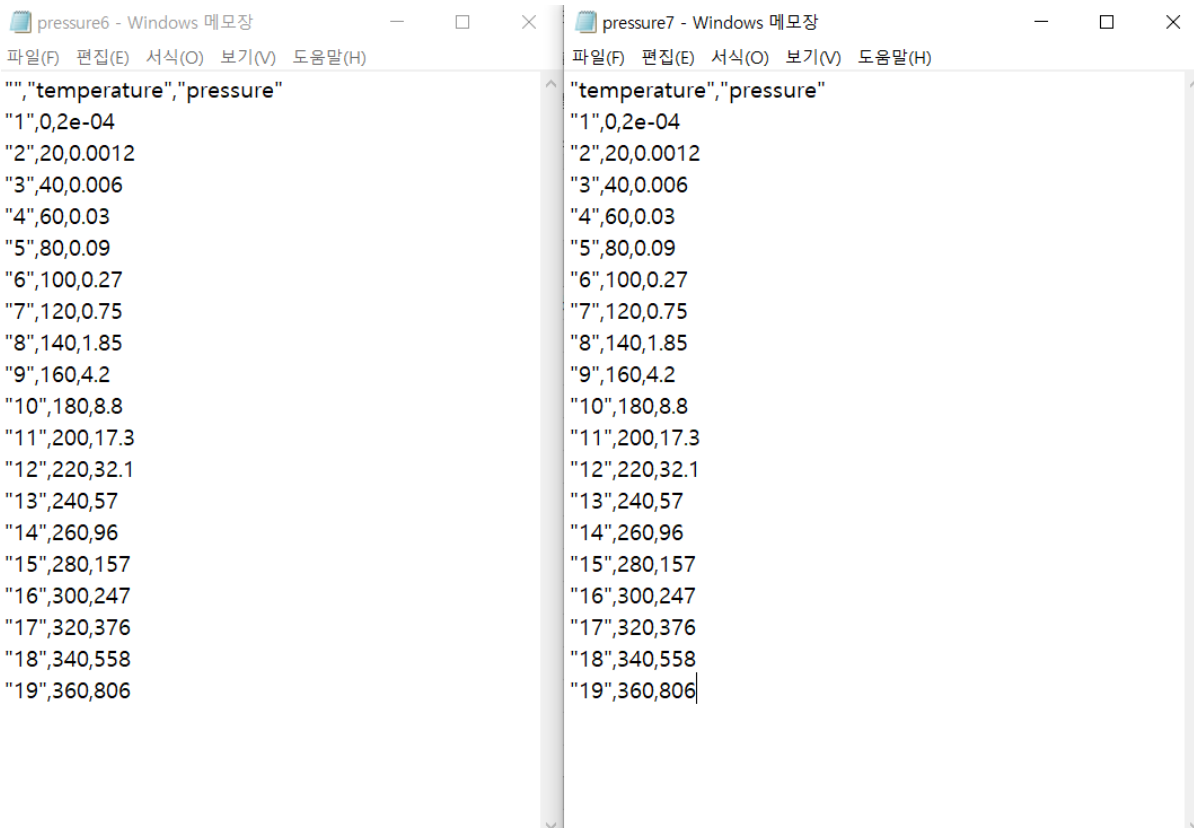
- write.csv() 함수는 기본적으로 write.table() 함수에서 sep = "," 옵션을 지정한 것과 동일한 결과를 생성

3. 데이터 저장하기

예제 2-17 R의 기본 패키지에 저장된 데이터 pressure를 write.csv() 함수를 이용하여 현 작업경로에 외부 파일로 저장하고 write.table() 함수를 이용하여 저장한 결과와 비교

```
> write.csv( pressure, "pressure6.txt" )
```

```
> write.table( pressure, "pressure7.txt", sep="," )
```



pressure6 - Windows 메모장

```

", "temperature", "pressure"
"1", 0.2e-04
"2", 20.0012
"3", 40.0006
"4", 60.003
"5", 80.009
"6", 100.027
"7", 120.075
"8", 140.185
"9", 160.42
"10", 180.88
"11", 200.173
"12", 220.321
"13", 240.57
"14", 260.96
"15", 280.157
"16", 300.247
"17", 320.376
"18", 340.558
"19", 360.806
                    
```

Ln 19, Col 13 100% Windows (CRLF) UTF-8

pressure7 - Windows 메모장

```

"temperature", "pressure"
"1", 0.2e-04
"2", 20.0012
"3", 40.0006
"4", 60.003
"5", 80.009
"6", 100.027
"7", 120.075
"8", 140.185
"9", 160.42
"10", 180.88
"11", 200.173
"12", 220.321
"13", 240.57
"14", 260.96
"15", 280.157
"16", 300.247
"17", 320.376
"18", 340.558
"19", 360.806
                    
```

Ln 20, Col 13 100% Windows (CRLF) UTF-8

<저장된 text 파일 확인>

3. 데이터 저장하기

- `write.csv(pressure, "pressure6.txt")` 명령은 기본적으로 구분 기호로 콤마를 사용하는 반면 `write.table(pressure, "pressure7.txt", sep = ",")` 명령은 이렇게 옵션을 명시적으로 지정해야 콤마를 구분 기호로 사용
- `write.csv()`는 `row.names = TRUE`와 `col.names = NA`를 기본값으로 사용.
첫 번째 열이 실제 데이터 열이 아니라 행 이름 열임을 나타내기 위해 헤더의 첫 칸을 비워 두는 역할
- `write.table()`은 `col.names = TRUE`가 기본이므로 헤더의 첫 칸을 비우지 않아 빈 머리글이 나타나지 않음

04

데이터 구조의 확인 및 결측치 지정

4. 데이터 구조의 확인 및 결측치 지정

▶ `str()` 함수를 이용하여 데이터 구조 확인하기

- R에서 불러온 자료구조를 확인하기 위해서는 `str()` 함수를 사용

```
> str(women)
```

```
'data.frame': 15 obs. of 2 variables:
```

```
$ height: num 58 59 60 61 62 63 64 65 66 67 ...
```

```
$ weight: num 115 117 120 123 126 129 132 135 139 142 ...
```

- `read.table()`, `read.csv()`, `data.frame()` 등에서 `stringsAsFactors = FALSE` 옵션을 사용하게 되면,
데이터에 문자가 포함되어 있더라도 이를 범주형 변수인 팩터(factor)로 변환하지 않고 그대로
문자형(character)으로 인식하여 불러오게 됨

(R 4.0.0 이후 버전에서는 기본값이 이미 FALSE로 변경되어 별도의 지정 없이도 문자형이 유지되지만,
구버전에서는 기본값이 TRUE였기 때문에 문자형을 유지하려면 반드시 이 옵션을 명시해야 함)

4. 데이터 구조의 확인 및 결측치 지정

예제 2-18 read.csv() 함수를 이용하여 현 작업경로에 저장되어 있는 USArrests1.csv 데이터세트를 불러오되,

“stringsAsFactors=F”라는 옵션을 이용하여 문자변수가 Factor 변수로 인식되지 않도록 지정

```
> us_dat2 <- read.csv("USArrests1.csv",header=T,stringsAsFactors=F)
```

```
> str(us_dat2)
```

```
'data.frame': 50 obs. of 5 variables:
```

```
$ State : chr "Alabama" "Alaska" "Arizona" "Arkansas" ...
```

```
$ Murder : num 13.2 10 8.1 8.8 9 7.9 3.3 5.9 15.4 17.4 ...
```

```
$ Assault : int 236 263 294 190 276 204 110 238 335 211 ...
```

```
$ UrbanPop: int 58 48 80 50 91 78 77 72 80 60 ...
```

```
$ Rape : num 21.2 44.5 31 19.5 40.6 38.7 11.1 15.8 31.9 25.8 ...
```

▶ na.strings 옵션을 이용한 결측치 지정

- R에서는 결측치가 'NA'로 표시되지만, 데이터를 읽어 오면서 na.strings와 같은 옵션을 지정해 주면 특정한 문자를 결측치로 인식

4. 데이터 구조의 확인 및 결측치 지정

예제 2-19 작업경로에 'dat3_exam1.txt'라는 이름으로 저장되어 있는 아래 데이터를 불러올 때,
관측값 'aa'를 결측치로 인식하여 불러오기

```
var1 var2
```

```
1 11 aa
```

```
2 22 bb
```

```
3 33 cc
```

```
>                                nadat                                <-
```

```
read.table('dat3_exam1.txt',na.strings='aa',header=T)
```

```
> nadat
```

```
var1 var2
```

```
1 11 <NA>
```

```
2 22 bb
```

```
3 33 cc
```

05

객체 확인 및 삭제

5. 객체 확인 및 삭제

▶ ls() 함수를 이용한 객체 확인

- ls() 함수를 이용하면 현재까지의 작업 중 만들어진 객체를 모두 확인할 수 있음
 - sort(ls())를 사용하면 이름을 정렬해서, length(ls())를 사용하면 객체가 몇 개인지 조회할 수 있음
- ```
> ls()
```

```
[1] "a" "b" "dat2" "dat3" "dat4"
```

### ▶ rm() 함수를 이용한 모든 객체 삭제

- rm() 함수를 수행하면 생성된 모든 객체를 삭제할 수 있음
- character(0)은 “길이가 0인 문자형 벡터”를 의미하며 현재 작업 공간에 저장된 객체 이름이 하나도 없다는 것을 나타냄

```
> rm(list=ls())
```

```
> ls()
```

```
character(0)
```

06

실습하기

04강

다음시간안내 ▶▶▶

# R레이터구조 (1)